AGENTIC WORK BEST PRACTICES

Adaptive Agent Modes - Cost Ladder + Governance

Release 0.2.0 promotion payload

What this is. The adaptive mode ladder picks the least expensive scaffolding level that still carries the governance controls a task's consequences demand: a human gate on critical consequence, durable checkpoints and handoff for wide or multi-session work, and a spec-freeze that blocks silent scope-shrink on underspecified tasks.

What this is NOT. It is not a correctness-lift claim. The measured evidence is a decisive null: across 5 admission-gated fixtures and 10 paid calls, every arm - unscaffolded vanilla included - converged at 100/100 under fair semantic grading on single-pass-sized workspaces, while the scaffolded arms cost 5-8x the tokens (the Full arm ~5.2x the main vanilla arm on the v4 re-grade alone). What the scaffolding bought was not score but PROCESS artifacts: only the Full arm produced a complete impact map, and its "consumers verified" self-attestation was truthful (`self_attestation_gap = false`). Choose a mode to buy governance and auditability, not points. The numeric mode ceilings remain declared `hypothesis-pending-benchmark`.

Selection

The router chooses the cheapest mode covering four dimensions: **consequence** (blast radius - the only path to Full and its human gate), **breadth** (width of change) and **continuity** (session boundaries), which each cap at Mode 3, and **clarity** (an underspecified mutating task floors at Mode 2 and receives the spec-synthesis capability). Action intent is separate: explaining, planning or dry-running a deployment or credential rotation does not select Full.

Mode	Use	What you pay for	Typical capability profile
Vanilla	Advisory / read-only answer; no file changes	Nothing (no scaffold)	Fast provider profile
Mode 1 - Lean	Small, reversible, local change	Minimal Core, compact requirement ledger, focused verification	`fast-low-risk`
Mode 2 - Routed	Ordinary feature or bounded bug fix	Precision Context Pack, objective/evidence ledger, pre-submit gate	`balanced-daily`
Mode 3 - Assured	Public contract, open-world parser, security-sensitive local change, wide mechanical change, multi-session implementation	Impact map, adversarial checks, durable checkpoints and session handoff	`deep-implementation`
Full	Critical consequence only: production migration rollout, external side effect, credential rotation, cross-system release	Human scope approval, independent verifier, bounded loop	`architecture-high-risk` plus `adversarial-review`

Size never buys the human gate: a 30-file mechanical rename stays at Mode 3, while a one-line credential rotation is Full. Higher modes cost real tokens - budget them like any other spend.

Governance controls by mode

1. Mode 2+ requires a reproducible pre-submit PASS (`.agentic/run-pre-submit.ps1` / `.sh`); unresolved material ambiguity blocks completion.
2. Mode 3+ additionally gates six impact-map areas and an adversarial challenge; wide or multi-session work gates a durable checkpoint and session handoff.
3. Full also requires explicit human scope approval and an independent verifier PASS (the verifier is a separate call - Full is never one call).
4. Underspecified mutating tasks (clarity axis) must produce a validated spec (`spec.json` : surface inventory, convention inventory with file evidence, pinned decision points, risk register, acceptance tests) whose requirement ledger is FROZEN: scope may only shrink through a recorded `owner_scope_changes` entry; additions are always allowed. The pre-submit gate re-executes every acceptance test.
5. Decision-time research surfacing: design/benchmark/architecture prompts get the top topic-matched findings from `Research/knowledge-base/findings-index.json` attached to the routing result (`relevant_findings`) and the Context Pack.
6. Capability profiles resolve through the weekly expiring provider catalog (`Models/providers.json`); stale data blocks automatic selection.

Layout (additive - the classic router is untouched)

- `tools/route.ps1` - classic keyword router by default (unchanged behaviour); `-Adaptive` opts into the layer below.
- `tools/adaptive/` - the Python: `prepare_or_route.py` (entry), `adaptive_router.py` , `prepare_context.py` , `objective_compiler.py` , `pre_submit_gate.py` , `spec_synthesis.py` , `process_metrics.py` , `fixture_admission.py` , `claims_ledger.py` , `vault_hygiene.py` , `resolve_capability_profile.py` , plus `modules/` , `schemas/` and a local `findings-index.json` copy.
- `Runtime/adaptive-modes.json` - the five-mode taxonomy, axes, escalation triggers, capability profiles.
- `Router/catalog/adaptive-modules.json` - the precision knowledge catalog used ONLY by the adaptive path (`Router/catalog/modules.json` remains the classic catalog).
- `Evals/adaptive-fixtures/` - the admission-gated boundary fixtures; not

registered in the stable fixture catalog (see the README there).

- `tools/vault-hygiene.ps1`, `tools/spec-synthesis.ps1` - thin launchers.

Commands

```
# Classic (unchanged)
powershell -ExecutionPolicy Bypass -File tools/route.ps1 -Repo <repo> -Task "<task>"

# Adaptive: routing decision only (no writes)
powershell -ExecutionPolicy Bypass -File tools/route.ps1 -Adaptive -NoWrite -Repo <repo> -Tas

# Adaptive: compile the mode-specific context pack into <repo>/.agentic
powershell -ExecutionPolicy Bypass -File tools/route.ps1 -Adaptive -Repo <repo> -Task "<task>
powershell -ExecutionPolicy Bypass -File tools/route.ps1 -Adaptive -Repo <repo> -TaskFile <ta

# Completion gate (from the workspace)
powershell -ExecutionPolicy Bypass -File .agentic/run-pre-submit.ps1
```

The adaptive path needs `python` (or the Windows `py` launcher) on PATH and fails with a clear message when neither exists. `-ForceMode` exists only for scaffolded benchmark arms; normal work must use adaptive selection.

Benchmark rule

The coding benchmark baseline is a separate, unscaffolded task-only `vanilla-control`; it never reuses the read-only runtime Vanilla preset. Compare arms on the same admission-gated fixture, model and effort; fixtures without valid paid history get a one-call canary on the cheapest arm first, and the remaining arms need a separate approval constructible only from a valid canary run-record. Count the Full verifier call honestly. Given the measured null, prefer the CHEAPEST mode whose governance controls the task's consequences actually require; never argue a higher mode from expected quality lift without new fixture evidence that shows separation.